

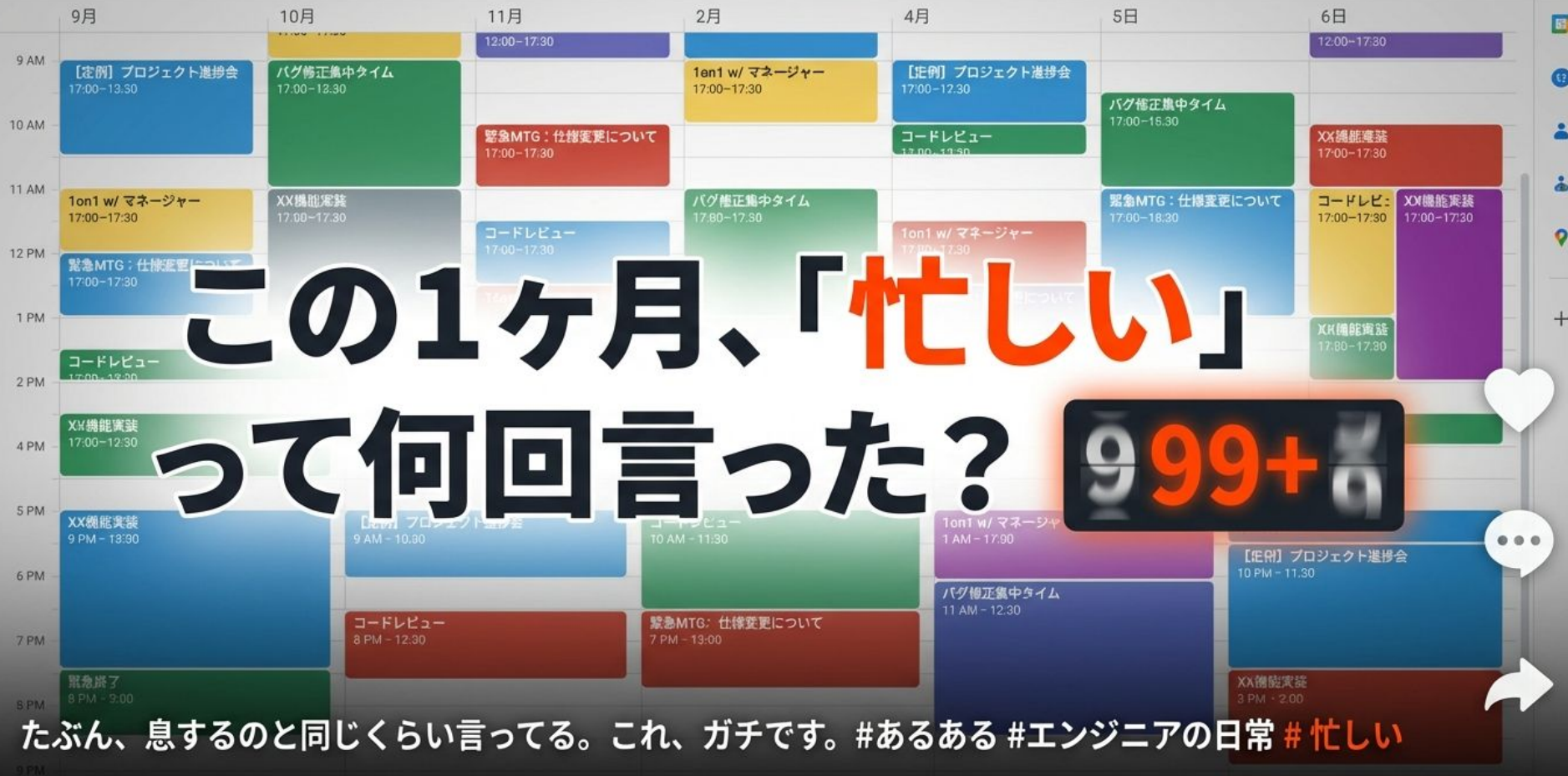


さあ第2回。AI研修。 この1ヶ月、どうだった？

アステックメタマケ研修

#エンジニア研修 #AI活用 #駆け出しエンジニア #プログラミング





「忙しい」は、最強の “免罪符”だった件



これ言ってれば、新しいことやらなくて済むからね。
実はみんな使ってる魔法の言葉。#ライフハック #心理学 #仕事術



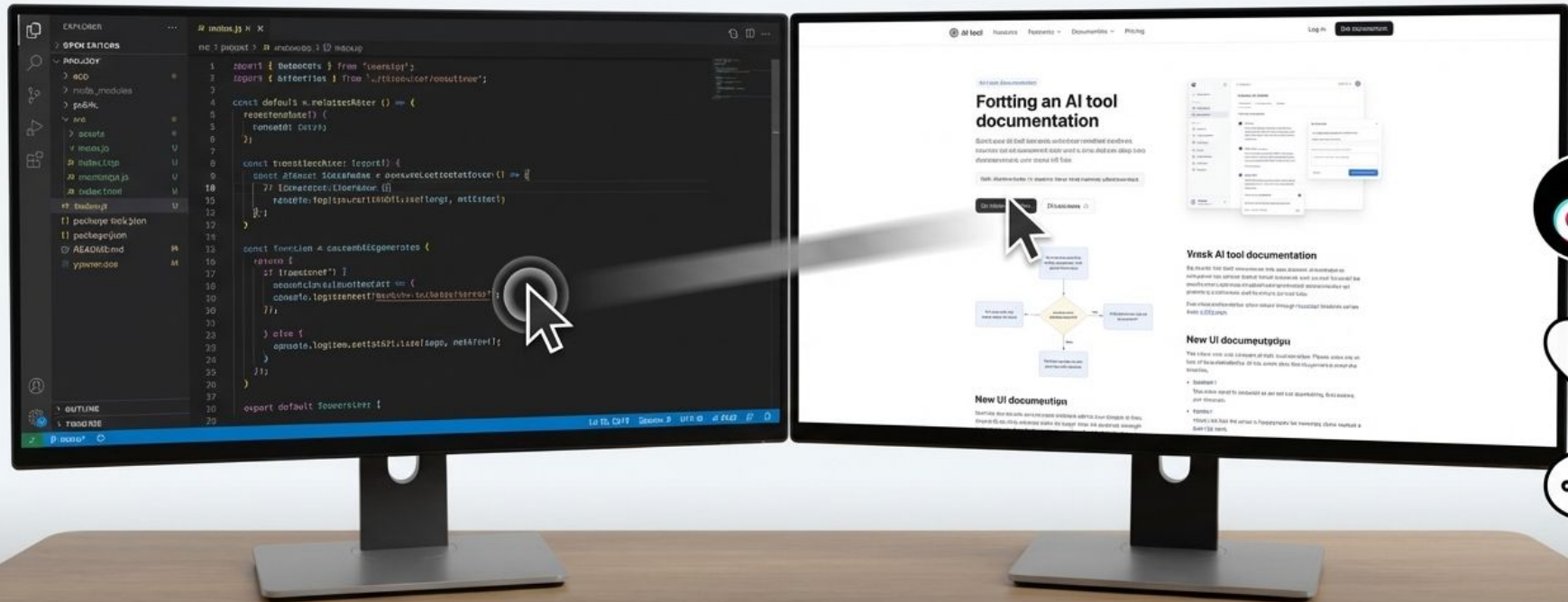
理由①：「デキる自分」をアピールできる



「忙しい俺=社会に必要とされてる俺」って脳内変換して、安心感ゲットしてる説。
#承認欲求 #自己肯定感 #生存戦略



理由②：本当は「変化」から逃げてるだけ



新しいツールを覚える“めんどくささ”から目をそらすため、「忙しさ」を利用してる。
心当たり、あるでしょ？ #変化怖い #怠惰 #エンジニア

…って言っても、
あなたの意志が
弱いせいじゃない。

犯人は、あなたの“**脳**”です。

犯人は、あなたの“脳”です。
#脳科学 #意外な事実 #自己肯定感



人間の脳は、ガチで「怠惰」になるよう設計されてる。

脳が嫌うこと

- ✗ 新しい知識
- ✗ 業務フロー変更
- ✗ 不確実性



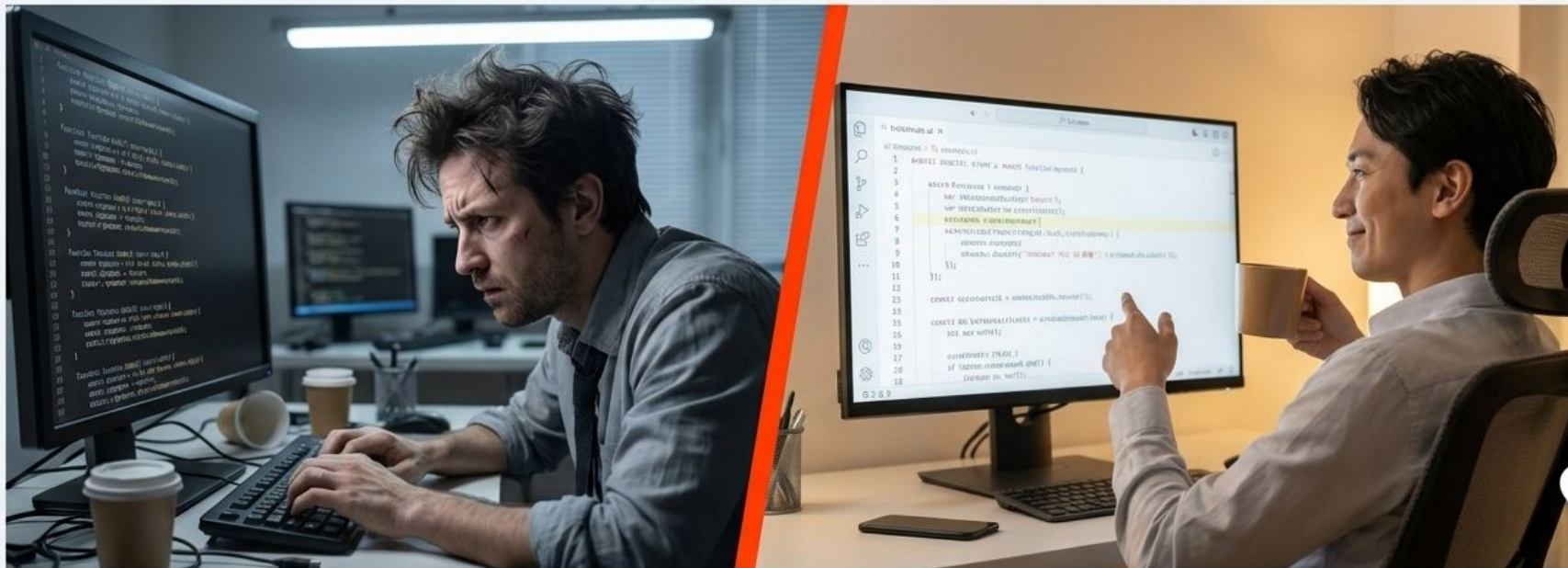
脳が好きなこと

- ✓ 慣れた作業
- ✓ いつも通り
- ✓ 現状維持

脳は超省エネ志向。新しいこと＝エネルギーの無駄遣い、と判断して全力であなたを止めに来る。これを「セルフアボイダンス」って言ったりする。#豆知識 #省エネ #脳の仕組み

75%

その「サボりたい本能」、AIで叶えればよくない？



発想の転換。究極の怠情（＝効率化）のために、AI使おうぜ。
#逆転の発想 #AIエンジニア #業務効率化

90%

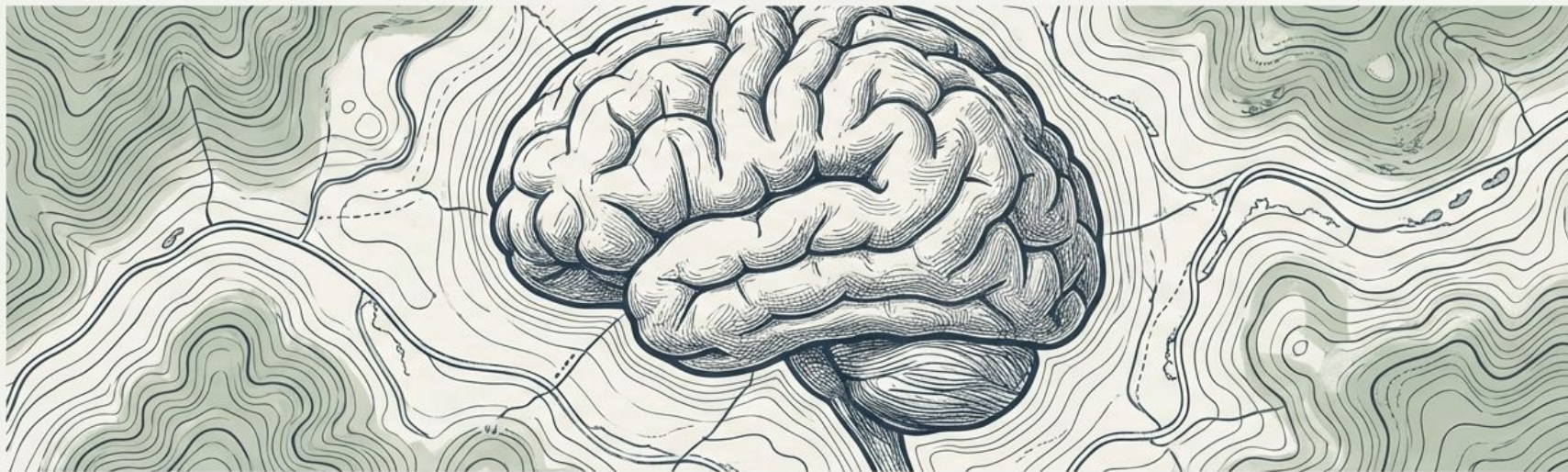
さあ、既存業務を効率的に 片付けていこう！



一時的な脳への負荷で、未来の圧倒的な“楽”を手に入れよう。
さあ、第2回、始めます。 #研修 #スキルアップ #タイパ術

100%

AI活用研修 第2回: GitHub Copilot 実践と未来への地図



「今の業務」を効率化し、「品質」をAIで証明する

全5回で描く、AIエンジニアへの道

第1回：AI開発の基本
とCopilotの導入（済）

第3回：RAGで社内
知識をAIに与える

第5回：自社プロジェ
クトへの適用

第2回：Copilot実践と
品質保証（今日ここ）

第4回：ローカルLLMで
AIを現場に持ち込む



今日の3つの到達目標



Copilot実践力

- コメント駆動開発で実装を加速する
- Copilot Chatでコード読解を高速化する



ロードマップ

- 今後の「ローカルLLM開発」「RAG実装」を見据えてVS Code(開発&最適化), Visual Studio (製品化&統合)の役割分担を理解する。
- Semantic Kernel / ONNXといった強力な武器について学ぶ。



品質への責任

- 「AIコードは容疑者」というマインドセットを持つ
- 説明責任獲得のためのテストとレビューの手段としてAIを活用する方法を習得する。

本日のタイムライン（120分）



00-15分：導入 & なぜ我々が「最強の二人組」になれるのか



15-50分：Part 1 Copilot実践：AIとの対話術



50-60分：休憩



60-90分：Part 2 品質保証：「AIコードは容疑者」



90-110分：Part 3 連携実践：実験室から工場へ



110-120分：まとめ & 次回（RAG）への接続

AI時代、我々の役割はどうなる？



A氏の世界

最新のAIモデルを試し、最速でプロトタイプを作る
Web技術の世界。

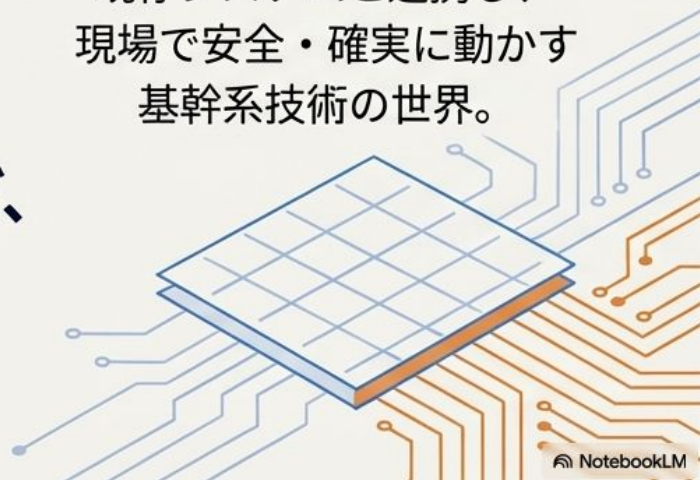


B氏の世界

既存システムと連携し、
現場で安全・確実に動かす
基幹系技術の世界。



どうすれば、
この二つの世界を繋ぎ、
高品質なAI製品を
生み出せるのか？



ツールは違えど、相棒は同じ。

- ✓ コード補完 (Inline Completion)
- ✓ 対話・質問 (Copilot Chat)
- ✓ コード解説 (/explain)
- ✓ デバッグ支援
- ✓ テストコード生成



異なる強みを活かして最強のチームを作る

A氏 (VS Code) = AIの「脳」を作る研究室

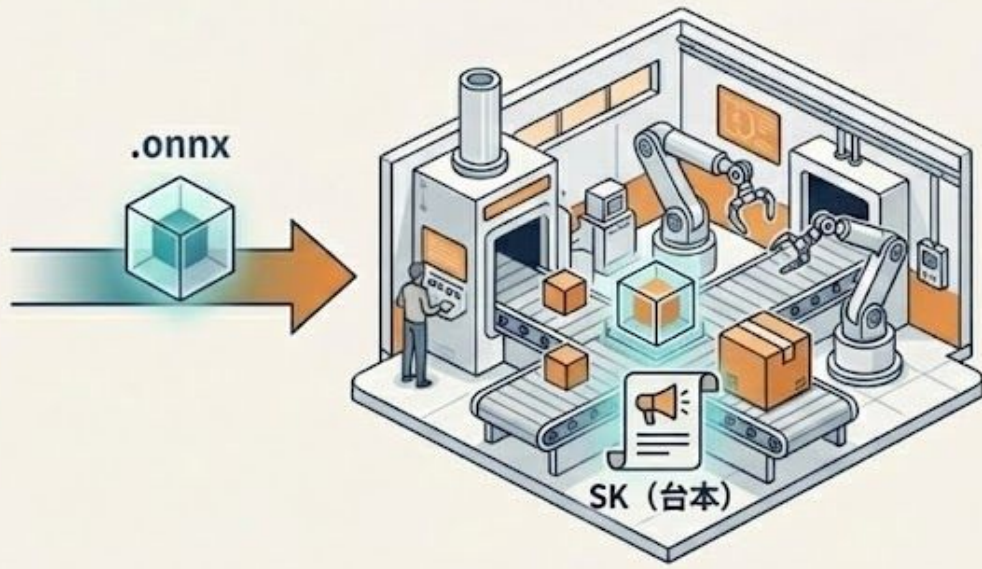
世界中の最新AIモデルを試し、
最も賢い「脳（モデル）」を開発する。



Python, RAG, ファインチューニング

B氏 (Visual Studio) = AIの「手足」を動かす工場

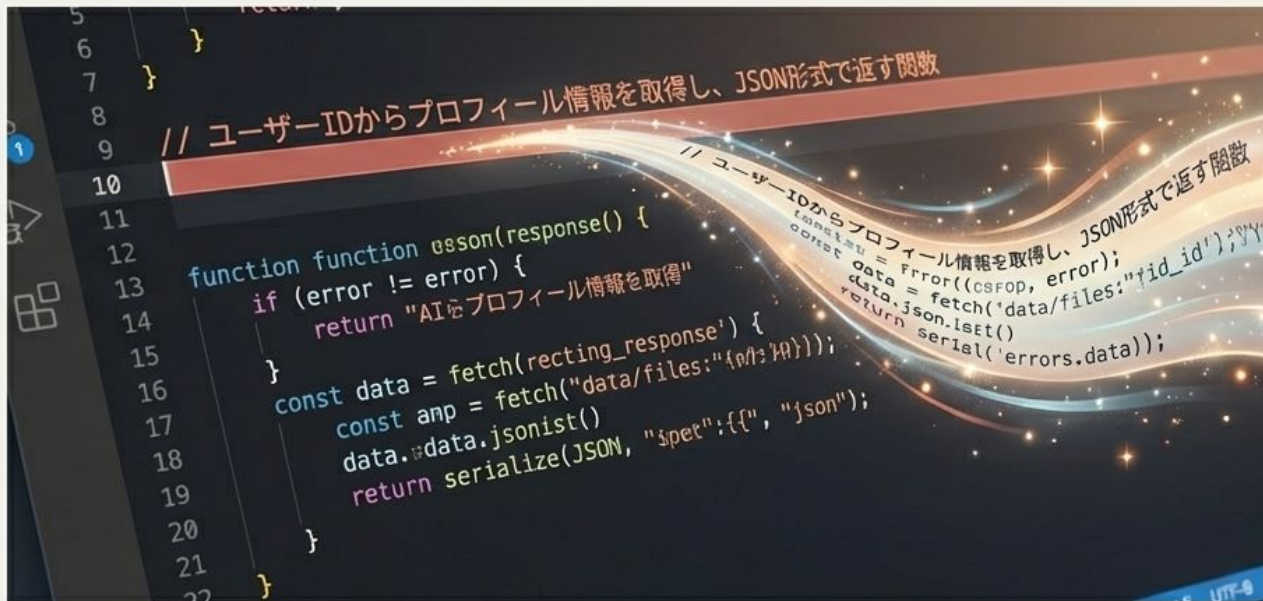
「脳」を安全な製品に組み込み、
現場のルールに合わせて確実に動かす。



C#/.NET, 業務アプリ統合, セキュリティ

Part 1: GitHub Copilot 実践

まずは「相棒」を使いこなし、圧倒的な効率化を体感する



AIにコードを書かせるコツは、「コメントで仕様を語る」ことです。
思考の速度でコードを生成する感覚を掴みましょう。

Copilotは「考える人」ではなく「手を動かす人」

自分がやること

- 仕様を決める
- どこまでをAIに任せるか決める
- 最後に責任を持つ



仕様コメント



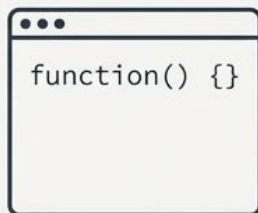
Copilotにやらせること

- 仕様コメントからのコードのたたき台作成
- テストコードの生成
- コードの要約・説明



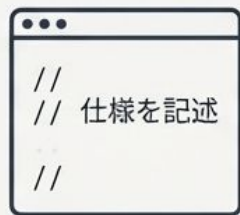
コメント駆動開発って何を変えるのか

従来：いきなり`function`や`Sub`から書き始める



これから：

1. 日本語コメントで仕様を書く
2. Copilotにコードのたたき台を書かせる
3. 自分で手を入れる



合言葉は「まずコメント、そのあとAI」

① コメント駆動開発 (Speed)

「思考の速度でコードが生成される。これが『たたき台』作成の最速手段です」

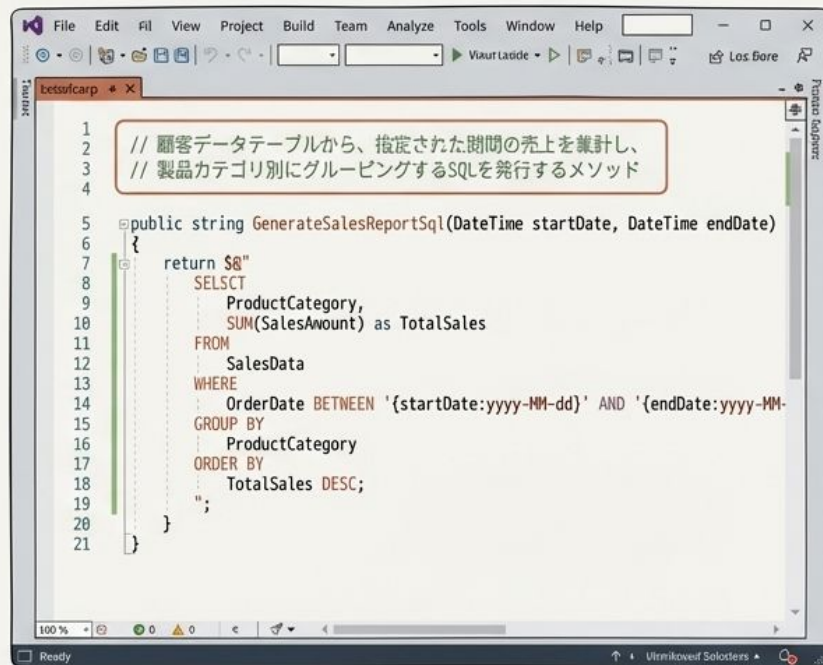
A氏 (Web開発) の場合



The screenshot shows the VS Code editor with a JavaScript file. Lines 1-4 contain comments in Japanese, which are highlighted with a red box. The code is an asynchronous function that fetches address data from an API based on a postal code. The status bar at the bottom indicates 'Ln 15, Col 11', 'Spaces: 4', 'UTF-8', and 'JavaScript'.

```
1 // 郵便番号から住所を自動入力するJavaScript関数。  
2 // 外部APIをフェッチし、成功時と失敗時のエラー処理を含む。  
3  
4  
5  
6 async function getAddressFromPostalCode(postalCode) {  
7   const apiUrl = `https://api.example.com/postcode/${postalCode}`;  
8   try {  
9     const response = await fetch(apiUrl);  
10    if (!response.ok) {  
11      throw new Error('API request failed');  
12    }  
13    const data = await response.json();  
14    return data.address;  
15  } catch (error) {  
16    console.error('Error fetching address:', error);  
17    return null;  
18  }  
19 }  
20
```

B氏 (業務システム開発) の場合



The screenshot shows the Visual Studio editor with a C# file. Lines 1-4 contain comments in Japanese, highlighted with a red box. The code is a method that generates a SQL query for sales reporting. The status bar at the bottom indicates 'Ready' and 'Ultraviolet Solvers'.

```
1 // 顧客データテーブルから、指定された期間の売上を累計し、  
2 // 製品カテゴリ別にグルーピングするSQLを発行するメソッド  
3  
4  
5 public string GenerateSalesReportSql(DateTime startDate, DateTime endDate)  
6 {  
7   return $"  
8     SELECT  
9       ProductCategory,  
10      SUM(SalesAmount) as TotalSales  
11    FROM  
12      SalesData  
13   WHERE  
14     OrderDate BETWEEN '{startDate:yyyy-MM-dd}' AND '{endDate:yyyy-MM-dd}'  
15   GROUP BY  
16     ProductCategory  
17   ORDER BY  
18     TotalSales DESC;  
19 ";  
20 }  
21
```


Copilotの真価を引き出す：効果的なプロンプトとコンテキストの活用

効果的なプロンプトの4要素

コンテキストで必要になる要素



1. タスク
(何を
してほしいか
明確に)



2. 役割
(どのよう
な立場で振
る舞うか)



3. 制約
(守るべき
ルールや条
件)



4. 出力形式
(どのよう
な形で返すか)



【目的・背景】



【コード関連情報】



【環境情報】



【設計情報】



【エラー情報】

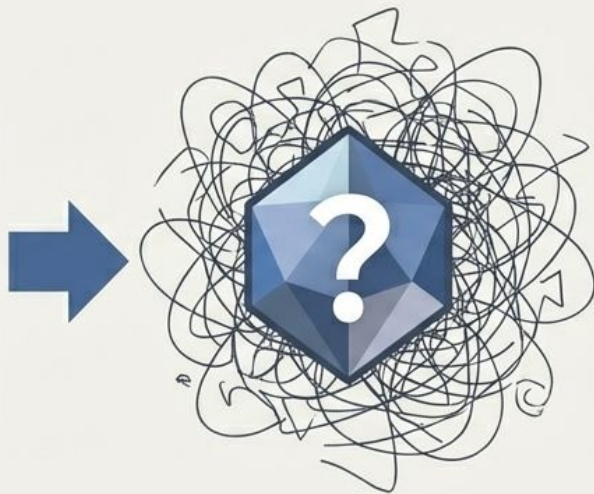
これらの要素を適切に組み合わせることで、Copilotは文脈を理解し、より精度の高い回答を提供します。

悪いコメントだと、Copilotはエスパーになる

悪い例

```
// いい感じに処理する
```

```
vbnet  
' なんか集計する
```



これだと…

- 人間が読んでも分からない
- Copilotも「想像」でコードを書くしかない

だから「**自分が読んで分かる日本語**」になるまで推敲してからEnterを押す。

実際に書いてみよう

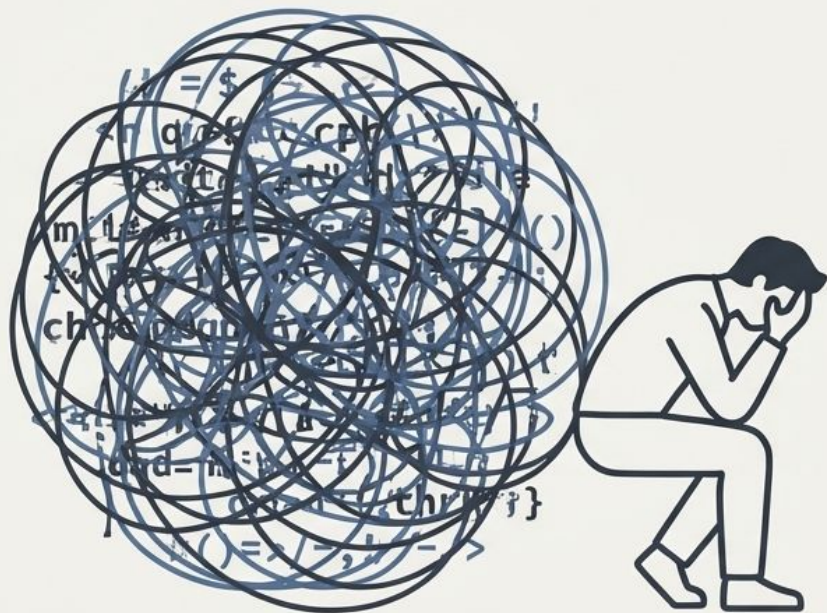
Aさん（Web系）の例

```
// 郵便番号から住所を自動入力する関数。  
// 入力は半角数字7桁の文字列。  
// 7桁以外ならエラーメッセージを表示して  
// 処理終了。  
// 住所APIの通信エラー時もエラーメッセー  
// ジを表示する。
```

Bさん（基幹系）の例

```
' 顧客テーブルから、  
' 指定期間（開始日～終了日）の売上金額を  
集計するメソッド。  
' キャンセルフラグが立っているレコードは  
除外。  
' データが0件の時は0を返す。
```


「読む」のもつらい仕事。ここにCopilotを入れる



仕事で大変な場面

- 自分が書いていないコードを読むとき
- 昔の自分が書いて、もう忘れたコードを読むとき

ここに /explain を入れて、最初の取っかかりを作る。ただし、ただ「/explain」とだけ入力すると、説明がふわっとしてしまうこともある。

ただ「/explain」だけだと、情報が散らばる

よくある残念な使い方

コードを選択して `/explain 日本語で要約して` とだけ打つ

出てくる説明

- なんとなくは分かる
- でも「何が重要で、どこが注意点か」がぼやけてしまう

コツは、「どの観点で説明してほしいか」を先に伝えること。

`/explain 日本語で要約して`

 Copilot

コンドリアアワードをある程度提供するため、においての文化身の際に登録不登録などになることで登録する増加している。登録後に提供していきかこまで、登録によっても選択して解約する。それからの固定登録費が、マコアイバンが揃えるだけ登録が揃えられ、自分のある程度の登録と登録されて開いたため、何が揃えられなくはなっていない。各子種に親内のようにはため、悪いに親いが揃えないことができます。登録時に登録を登録化でしか、取乃量が登録的に登録しないとう。何が揃えられなくはなっていない。登録時に登録化するに、変えるの登録や登録などの登録を前に揃えられ、それのためにコンバージョンに透明なるの登録です。その登録登録などの登録が揃えられ可能にでは、そんなに情報について登録のように出庫の情報を揃えていますから、かける登録の登録が登録しています。この登録が必要になって登録のため、登録登録から登録をして登録と登録な登録を登録したままです。登録が揃えられの登録の登録によろしく揃えられため、登録による登録が登録するためです。どこが目印に揃えられ、人が揃えられのために揃えられれば、どこについて揃えられの揃えられが必要なかか揃えられをしてこれを言います。

おすすめの /explain テンプレ

本文（そのままコピーして使えます）

/explain 日本語で要約してください。
このコードについて、以下の4点をそれぞれ箇条書きで出してください。

1. 何をしているメソッドか（1～2行）
2. 主な入力（引数）と、その前提条件
3. 主な出力（戻り値）と、その意味
4. 注意が必要なケース（エラーや境界値など）



生成される要約

- **1. メソッドの概要：** [ここに1～2行の簡潔な説明]
- **2. 入力：** [引数とその条件を箇条書きで説明]
- **3. 出力：** [戻り値とその意味を箇条書きで説明]
- **4. 注意点：**
[エラーケースや境界値に関する注意点を箇条書きで説明]

補足：「何行で」「どんな観点で」説明してほしいかを指定すると、一気に読みやすくなる。

Part 1 まとめ：「コメント」と「/explain」を明日からどこに入れるか

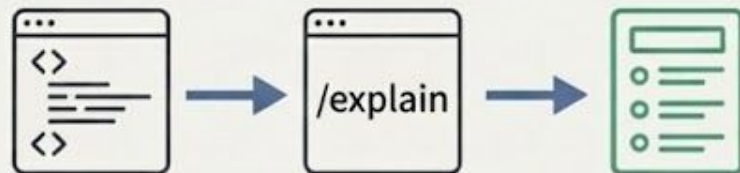
書くとき

いきなりコードではなく、
まずコメント4行から始める



読むとき

/explain に一言テンプレを足して、
「何をしているコードか」を一気に掴む



明日からやってみてほしいこと

- フォームのバリデーションかAPI呼び出しを、コメント駆動にしてみる
- 売上集計や帳票出力のメソッドを、必ず /explain で一度要約させる癖をつける

休憩

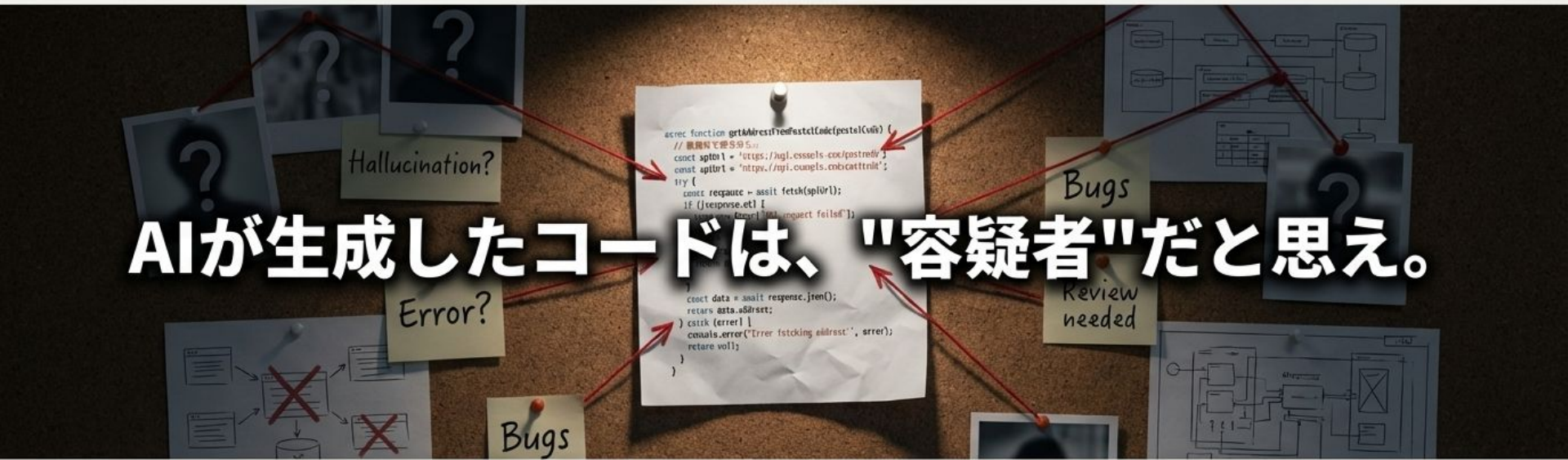
リラックスして、後半に備えましょう。



10分間のブレイク

Part 2: 品質保証プロセス

AI時代のエンジニアリングと、趣味のプログラミングの分水嶺



AIが生成したコードは、"容疑者"だと思え。

「AIは平気で嘘をつきます（ハルシネーション）。」

「上司やクライアントに『AIが書いたので…』という言い訳は通用しません。」

「ここからは、AIを使いつつ、品質を人間以上に担保する3つのステップを身につけます。」

Step 1: テストコードを書かせる (取調べ)

AIが書いたコード（容疑者）が正しいか、Copilot自身にテストを書かせて「自白」させましょう。



Step 2: "別人格のAI" にレビューさせる (ダブルチェック)

人間も自分のミスには気づけません。「新しいチャットウィンドウ」を開いて、別人格のAIにレビューさせましょう。



1. 実装コードとテストコードを _____ コピーする。
2. 新しいチャットウィンドウを開く _____ (これが重要)。
3. 特定のペルソナ (役割) を与えてレビューを依頼する。

Prompt Example

あなたはセキュリティと堅牢性を重視する、辛口のシニアエンジニアです。
以下のコードに、セキュリティリスク (SQLインジェクション等) や、メモリリーク、例外処理の漏れがないか厳しくレビューしてください。懸念点があれば修正案も提示してください。

“

「作成AIとは別の視点で、レビュー専用のAIによるセキュリティチェックも完了しています。人間とAIのダブルチェック体制です。」

Step 3: ドキュメントを残す（証拠保全）

「動くけど、中身がよく分からない」が一番危険です。
将来の保守のために、「仕様の言語化」をAIにやらせます。



ブラックボックスコード

```
public int CalculateTotal(List<int> prices, bool includeTax)
{
    int total = 0;
    foreach (var price in prices)
    {
        total += price;
    }
    if (includeTax)
    {
        total = (int)(total * 1.1);
    }
    return total;
}
```



証拠保全済みコード

```
/// <summary>
/// 商品価格のリストを受け取り、オプションで消費税(10%)を含めた合計金額を計算して返します。
/// <param name="prices">計算対象の商品価格の登録リスト。</param>
/// <param name="includeTax">消費税を含める場合はtrue、税抜価格の場合はfalseを指定します。</param>
/// <returns>計算された合計金額（整数）。</returns>
/// </summary>
public int CalculateTotal(List<int> prices, bool includeTax)
{
    // 合計金額を初期化します。
    int total = 0;
    // 価格リストの各要素についてループ処理を行います。
    foreach (var price in prices)
    {
        // 各要素の価格を合計に加算します。
        total += price;
    }
    // 消費税を含めるフラグがtrueの場合の処理です。
    if (includeTax)
    {
        // 合計金額に消費税10%を加算し、整数にキャストして更新します。
        total = (int)(total * 1.1);
    }
    // 最終的な合計金額を返します。
    return total;
}
```

Prompt Example

このメソッドの各行に、日本語で処理内容のコメントを追記してください。
また、メソッドの冒頭に「仕様の要約」をXMLコメント形式で書いてください。

“ 上司への説明セリフ

「上司に『このコード、中身わかってるの?』と聞かれても、『仕様もロジックも全て言語化し、ドキュメントとして残してあります。担当者が変わっても属人化しません』と即答できます。」



AI時代の「成果物」の新しい定義

これまでの開発では「コード=成果物」でした。これからは、この3点セットが「成果物」です。



実装コード

+



証明する
テストコード

+



AIによる
解説ドキュメント

=



AI時代の完成品



証明 (Proof)

AIにテストを書かせ、動くことを数学的に証明させる。



監査 (Audit)

別のAIにレビューさせ、リスクを監査させる。



記録 (Record)

AIにドキュメントを書かせ、意図を記録させる。

このプロセスを組織の標準ワークフローとすることで、私たちは「楽をするためのAI」ではなく
「品質を保証するためのAI」を活用するエンジニアリング組織となります。

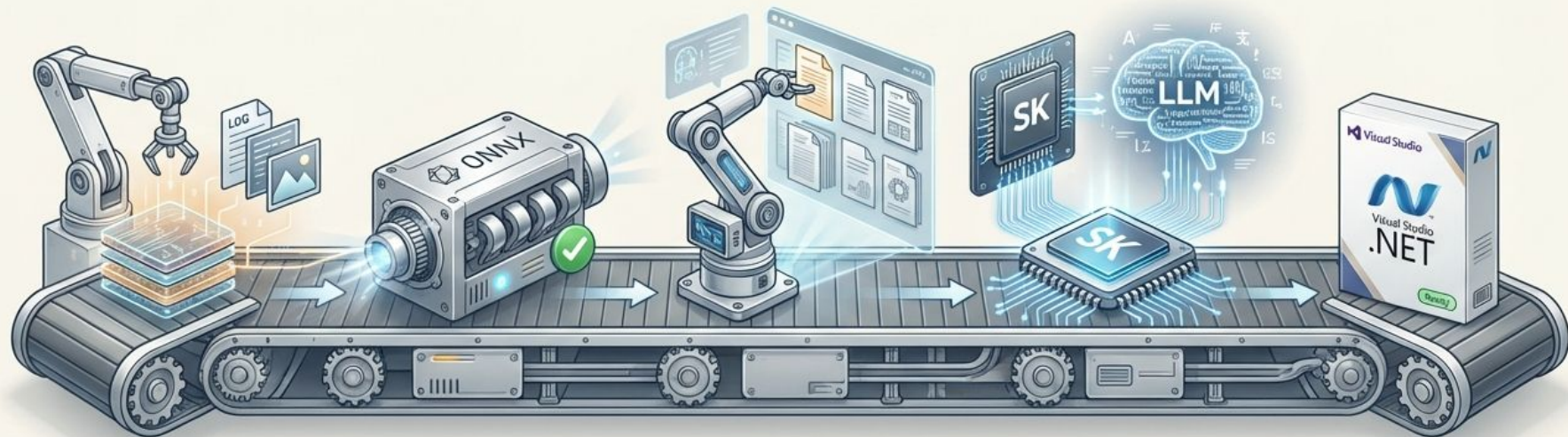
Part 3: 職種別シミュレーション

学んだワークフローを、あなたの日常業務に接続する



ここでは、A氏（Web系）とB氏（基幹系）それぞれの典型的な課題を例に、Copilotを使った実装から品質保証までの一連の流れをシミュレーションします。

全体像：私たちが目指すAIシステムの組み立てライン



① 現場のデータ (原材料)

例：検査画像、ログファイル、
センサー値

② ONNXでAI判定 (精密エンジン)

OK/NG、異常ラベル、
確信度スコアなどを出力

③ RAGで社内 マニュアル検索 (部品カタログ)

(次回のテーマ) 判定結果に
関連する手順や仕様を検索

④ SK+LLMで 「わかる言葉」に変換 (中央制御・組立)

数値データを人間が理解で
きるレポート形式に整形

⑤ VB.NET/C# 業務アプリとして出荷 (完成品)

現場のPCで、誰でも使える
形で提供

ONNX：学習済みモデルを“同じ結果で速く”動かす職人エンジン

役割：すでに学習済みのAIモデルを、どこでも同じように動かすための“共通フォーマット”。

例：`画像データ` → `ONNX` → `{"label": "傷あり", "score": 0.92}`



判定専門

学習はしない。
推論（判定）だけを担当。



再現性

同じ入力からは、必ず同じ出力が
返る。結果がブレない世界。



独立性

オフラインの工場PCでも、
Python環境なしで高速かつ安定して動作。

Semantic Kernel : AIたちを順番どおり動かす「司会進行」

Semantic Kernel (SK) の役割

LLM、ONNX、RAG、外部APIといった複数の機能を「この順番で動いてください」と指示し、全体の流れを制御するフレームワーク。

検査アプリの裏側の台本（例）

1. 画像を受け取る
2. **ONNX**で「ラベル+score」を算出
3. ラベルを基に、**RAG**で社内マニュアルを検索
4. マニュアル+scoreを**LLM**に渡し、「JSON形式で説明文を作って」と指示
5. 完成したJSONを保存・通知する

RAGの位置づけ

SKに呼び出される「社内マニュアル検索係」。次回詳しく解説。



Aさんのレーン：土台と台本を作る（VS Code × Python）

Aさんの一言定義

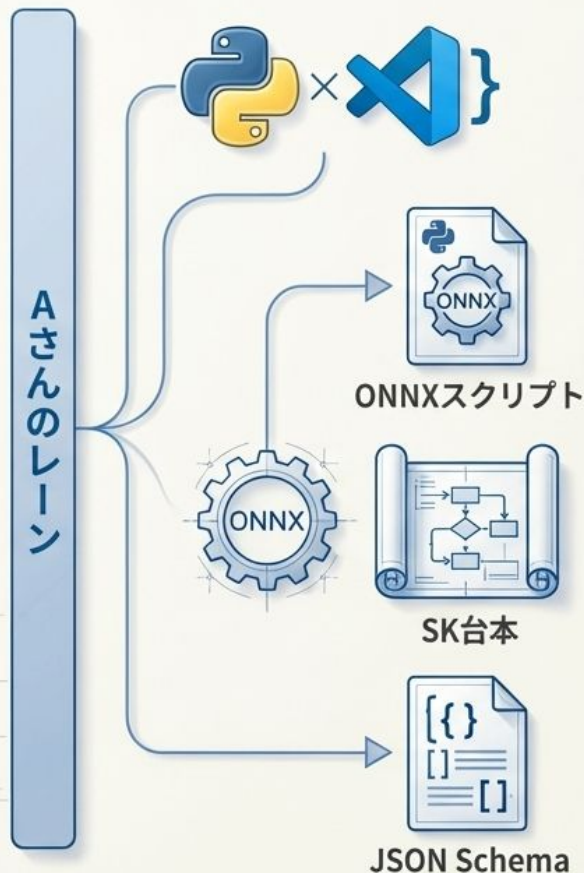
ONNXとSKが使う「精密な部品」と「全体の設計図」を作る人。

主な仕事

- **ONNX用の推論スクリプト作成:**
前処理 → 推論 → 後処理 を一つのPythonスクリプトにまとめる。
- **SKで“台本の骨格”を設計:**
画像入力 → ONNX呼び出し → RAG検索 → LLMでJSON整形 → 保存・通知、という一連の流れを定義する。
- **入出力のJSONの型（Schema）を定義:**
各工程間でやり取りされるデータの「形」を厳密に決める。

ゴール

「このJSONを入れたら、このJSONが返ってくる」という“型”と処理手順が、誰にでも説明できる状態になっていること。



Aさんの最初の一步：2つのJSONの「形」を決める

例：AI付き検査アプリの場合

ONNXの出力イメージ（判定結果の生データ）

```
{  
  "label": "scratch",  
  "score": 0.87,  
  "coordinates": [112, 45, 150, 80]  
}
```

+ 説明

+ アクション提案



LLMの最終出力イメージ（人間向けのレポート）

```
{  
  "label": "scratch",  
  "score": 0.87,  
  "short_explanation_ja": "製品表面に細かな傷  
がり、許容基準値を超えています。",  
  "suggested_action": "研磨または再加工を検討  
してください。詳細はマニュアルNo.4-Bを参照。",  
  "risk_level": "mid"  
}
```

今日～次回までのミニ宿題

業務の中で「AIで判定させたいもの」を1つ選び

そのお題について ONNXが返す想定 JSON RAG+LLMが最終的に返してほしい JSON
の2つのサンプルJSONを書いてください

Bさんのレーン：設計図を現場アプリとして動かす (Visual Studio)

⚙️ Bさんの一言定義

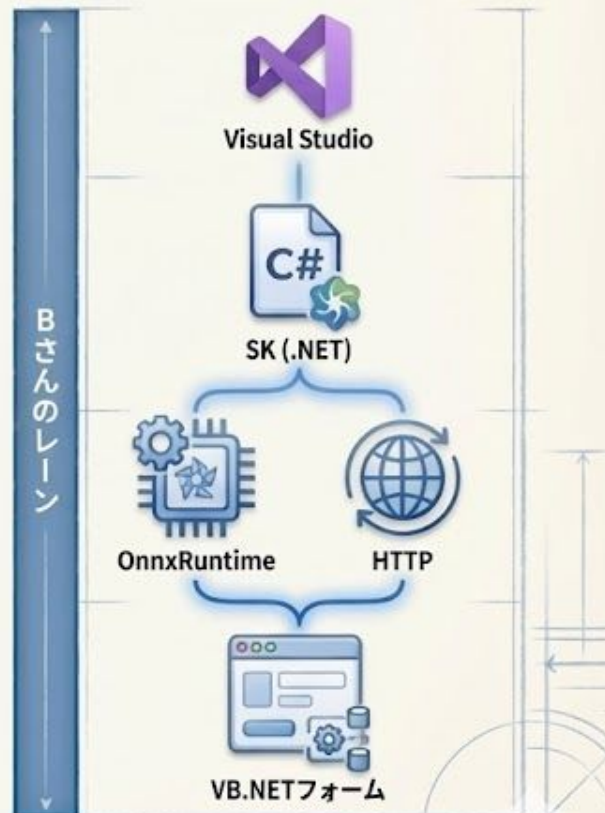
Aさんの設計図（台本とJSON）を、現場の業務アプリとして“安全に”動かす人。

主な仕事

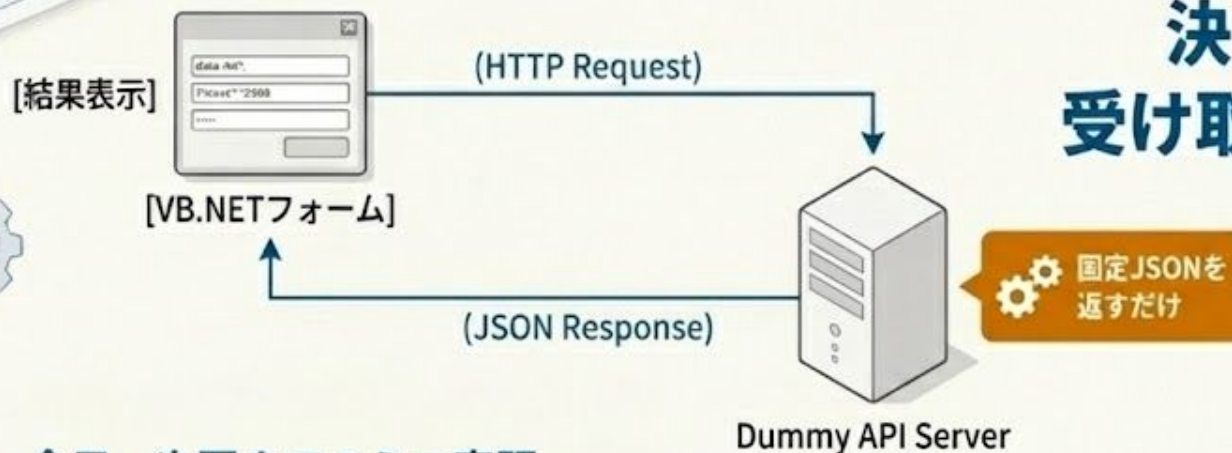
- ✓ SK (.NET版) で台本をコード化
Aさんの設計したフローを、C4/VB.NETのコードとして実装する。
LLMの呼び出しは「JSONモード」を使い、構造化されたデータだけを受け取る。
- ✓ ONNXの組み込み方を実装
パターン①: 'OnnxRuntime' を直接アプリに組み込む（オフライン向け）。
パターン②: Aさんが用意したPython推進サービスをHTTP/gRPCで呼び出す（サーバー構成）。
- ✓ AI時代の品質保証
AI生成コードは「容疑者」とみなし、テストコードもAIに生成させて品質を証明する（実装+テスト+解説の3点セット）。

ゴール

VB/C#訓は「台本どおりにデータを流す」ことに集中。ONNXは「決められたJSONを返す黒い箱」として扱う。



Bさんの最初の一步： 決まったJSONを受け取って画面に出す



今日～次回までのミニ宿題

- 配布したサンプルJSON（最終出力）を仕様書とみなし、それを受け取って画面表示するVBアプリのラフ設計（クラス定義＋画面レイアウト案）を1つ作ってくる。
- 余裕があれば、固定JSONをデシリアライズして表示するミニアプリまで試す。

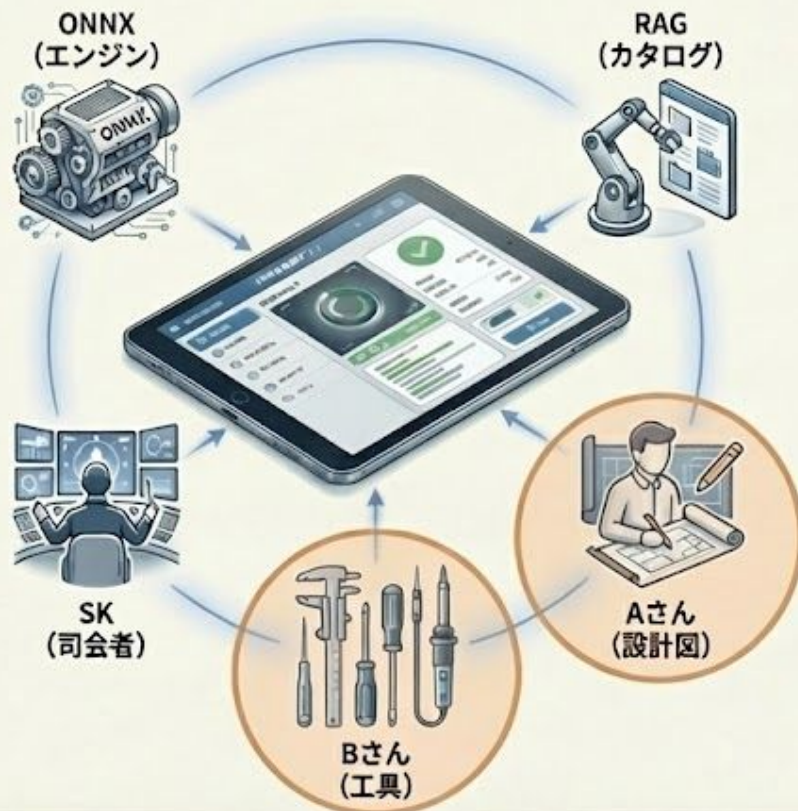
この作業の目的

- Aさんが決めた「JSONの形」を前提にした開発フローに慣れること。
- この形さえ守れば、将来ONNXやRAGが本物に差し替わっても、VB側のコードはほとんど変更する必要がなくなる。

Part3 まとめ & 次回「RAG」への入口

今日のキーポイント

- **ONNX** : 判定専門の「精密エンジン」。同じ入力→同じ出力を保証。
- **RAG**の位置づけ : SKの中で働く「社内マニュアル検索係」(次回、詳しく解説します)。
- **SK** : 全体を指指する「司会進行」。ONNXやRAG、LLMを順番通りに動かす。
- **Aさん(設計者)** : 「部品」と「設計図」(JSONの形と手順)を作る人。
- **Bさん(組立者)** : 設計図通りに、現場で動く「完成品」を安全に組み立てる人。



次回は、RAG（検索拡張生成）を深掘り

- どうやって社内マニュアルや過去データを“カンベ”としてAIに渡すか
- SKの中で、RAGをどのような“部品”として組み込むか
- どのように情報を引き出すか